parse_fastq.py 사용법과 설명

lab11/src/parse_fastq.py는 FASTQ 파일을 읽어 read 개수와 첫 번째 read의 정보를 출력하는 스크립트이다. 표준 라이브러리(sys)만 사용한다.

1. 사용법

lab 폴더(lab11/)에서 FASTQ 파일 경로를 인자로 주고 실행한다.

```
cd lab11
python3 src/parse_fastq.py data/SRR1553607_1.fastq
```

인자	설명
sys.argv[1]	읽을 FASTQ 파일 경로 (필수)

실행 결과 예시

```
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열:
GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACGCTCAAGCGAGATGATGGATATTTTTGAACGACTCA
첫 read 길이: 101
```

data/SRR1553607_1.fastq는 813,780줄이므로 813,780 ÷ 4 = 203,445개의 read가 들어 있다.
paired-end의 두 번째 파일(data/SRR1553607_2.fastq)도 같은 방법으로 실행한다.

2. FASTQ 형식

FASTQ는 read 하나를 4줄로 저장한다.

```
@SRR1553607.1 1 length=101        <- 1줄: '@' + read 이름
GTTAGCGTTGTTGATCGCGACGCAAC...     <- 2줄: 염기 서열
+SRR1553607.1 1 length=101        <- 3줄: '+' (이름 반복은 선택)
BB@FFFFFHHHHHJJJJJJJJJJJJ...     <- 4줄: 품질 점수 (서열과 길이가 같음)
```

품질 문자는 Phred+33 인코딩이다. 문자의 ASCII 값에서 33을 빼면 Phred 점수가 된다. 예를 들어 J는 74 - 33 = 41이다.

3. 코드 설명

read_fastq(filename)

FASTQ 파일을 읽어 (이름, 서열, 품질) 튜플의 리스트를 반환한다.

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()          # 파일 전체를 줄 단위 리스트로 읽음

    for i in range(0, len(lines), 4):  # 4줄씩 건너뛰며 read 하나씩 처리
```

```
        name = lines[i].rstrip()[1:]   # 1줄: 줄바꿈 제거 후 맨 앞 '@' 제거
        seq = lines[i + 1].rstrip()    # 2줄: 서열
        qual = lines[i + 3].rstrip()   # 4줄: 품질 (3줄 '+'는 사용하지 않음)
        records.append((name, seq, qual))

    return records
```

항목	내용
입력	FASTQ 파일 경로 (문자열)
반환값	[(name, seq, qual), ...] 형태의 리스트
name	@를 뺀 헤더 줄 전체 (예: SRR1553607.1 1 length=101)
seq	염기 서열 문자열 (대소문자 변환 없음)
qual	품질 문자열

반환된 리스트에서 reads[0]은 첫 번째 read이고, reads[0][0], reads[0][1], reads[0][2]는 각각 이름, 서열, 품질이다.

메인 부분

```
reads = read_fastq(sys.argv[1])
print("read 개수:", len(reads))
print("첫 read 이름:", reads[0][0])
print("첫 read 서열:", reads[0][1])
print("첫 read 길이:", len(reads[0][1]))
```

명령줄 인자로 받은 파일을 읽고 다음 네 가지를 출력한다.

1. 전체 read 개수
2. 첫 read의 이름
3. 첫 read의 서열
4. 첫 read의 길이(염기 수)

4. 주의사항과 한계

- 메모리 사용: readlines()는 파일 전체를 메모리에 올린다. 실습 데이터(약 20만 read)에는 문제가 없지만, 수 GB 크기의 FASTQ에는 한 줄씩 읽는 방식이 적합하다.
- 형식 검사 없음: 모든 read가 정확히 4줄이라고 가정한다. 줄 수가 4의 배수가 아니면 마지막 read에서 IndexError가 난다. 1줄이 @로 시작하는지, 3줄이 +로 시작하는지도 확인하지 않는다.
- 인자 누락: 파일 경로 없이 실행하면 sys.argv[1]에서 IndexError가 난다.
- 빈 파일: read가 하나도 없으면 reads[0]에서 IndexError가 난다.
- 압축 파일 미지원: .fastq.gz는 직접 읽을 수 없으므로 gunzip으로 먼저 압축을 풀어야 한다.
- 함수 재사용: read_fastq 아래의 메인 코드가 if __name__ == "__main__": 블록 안에 있지 않다. 그래서 다른 스크립트에서 from parse_fastq import read_fastq로 가져오면 메인 코드도 함께 실행된다.

5. 활용 예

read_fastq의 반환값으로 간단한 통계를 더 계산할 수 있다.

```
lengths = [len(seq) for name, seq, qual in reads]
print("평균 길이:", sum(lengths) / len(lengths))

first_qual = reads[0][2]
scores = [ord(c) - 33 for c in first_qual]   # Phred+33 -> 점수
print("첫 read 평균 품질:", sum(scores) / len(scores))
```